

Documentação ferramenta solidity para RPC

Murilo de Souza Neves

July 2026

1 Contrato Solidity

1.1 Classes de abstração de dados do contrato

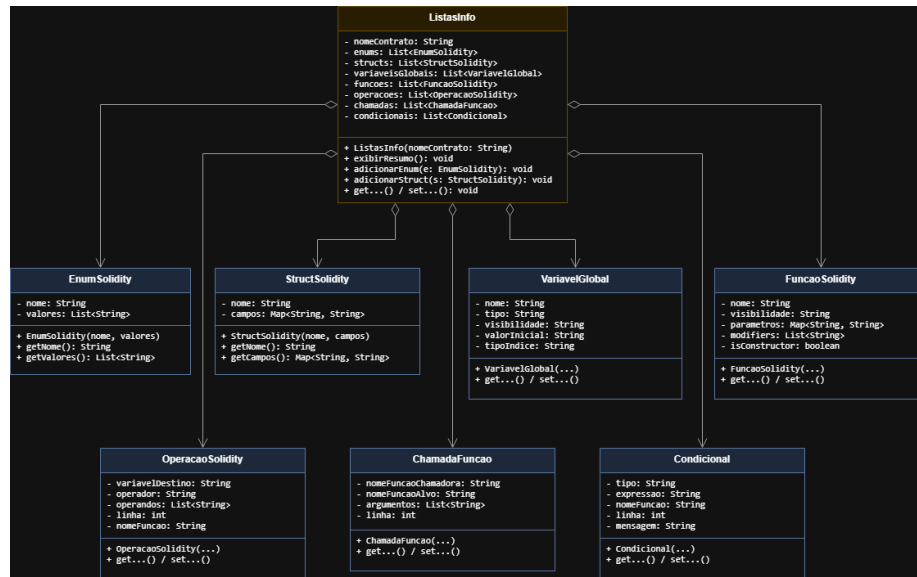


Figure 1: UML das classes de abstração de dados do Solidity

1.2 Classes de abstração de elementos da Rede de Petri

2 Leitura do contrato usando ANTLR

A classe de leitura do contrato responsável por identificar e extrair os objetos de estudo é a *SolidityVisitor.java*, a qual estende o padrão de projeto *Visitor* do ANTLR4. Ela possui três atributos de controle de estado: *info*, do tipo *ListasInfo*; *contratoAtual*, do tipo *String*; e *funcaoAtual*, também do tipo *String*.

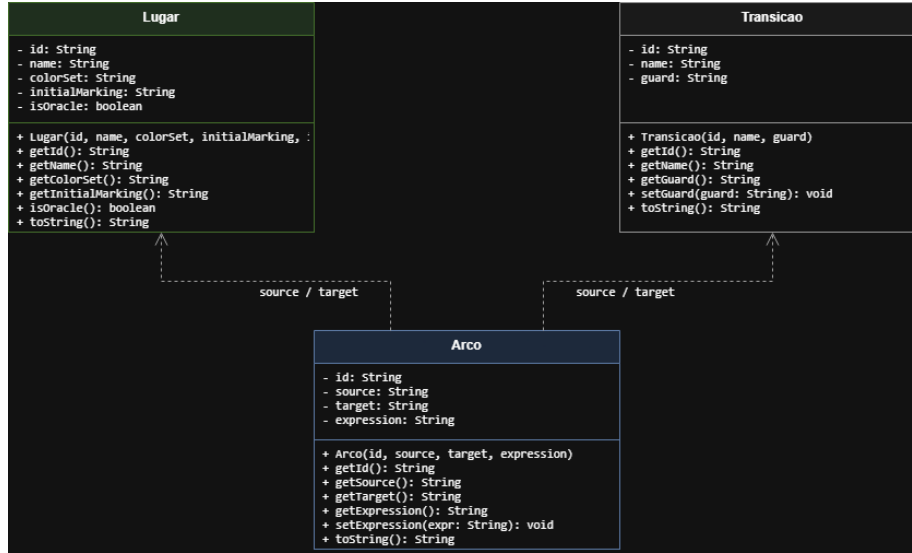


Figure 2: UML das classes de abstração de dados da Rede de Petri

A leitura do contrato se dá percorrendo a árvore sintática abstrata (AST) gerada pelo ANTLR, iniciando pela função *visitSourceUnit*, que inicializa o atributo *info* e propaga a travessia aos nós filhos do arquivo. Em seguida, aciona-se a função *visitContractDefinition*, responsável por isolar o escopo do contrato inteligente e identificar variáveis de estado, funções, construtor, *enums* e *structs*, delegando a extração de cada elemento aos seus respectivos métodos tratadores.

A função *visitStateVariableDeclaration* extrai o nome, o tipo base, a visibilidade e o valor inicial das variáveis globais do contrato. O método aplica filtros textuais para ignorar palavras-chave acessórias (como *constant*, *immutable* ou *override*) e captura a atribuição inicial, caso exista. Para a resolução de tipagem, aciona os métodos auxiliares *extrairTipoValor* e *extrairTipoIndice*, os quais retornam, respectivamente, o tipo do dado armazenado e, em caso de estruturas indexadas (*mappings* ou *arrays*), o domínio ou tipo de chave de índice da variável.

A função *visitConstructorDefinition* é dedicada à captura exclusiva do construtor do contrato. O método inicializa as listas de retorno e de modificadores como vazias, uma vez que o construtor do Solidity não os possui, extrai os parâmetros de entrada e seus respectivos tipos, registra as linhas de início e fim do bloco e, por fim, processa a lógica interna acionando o método *analisarBloco*.

O método *visitFunctionDefinition* realiza a leitura das funções padrão do contrato. Ele identifica o identificador da função, sua visibilidade, seus parâmetros de entrada e modificadores de acesso (como *onlyOwner*), acionando em seguida o *analisarBloco* para interpretar a semântica imperativa do bloco de execução.

A função *analisarBloco* identifica lógica interna das funções e do construtor

tor. Ela analisa o escopo delegando a leitura de estruturas específicas para suas respectivas funções: a leitura de condicionais é feita por *extrairCondicionais*, de mutações de estado com *extrairOperacoes* e de invocações intra-contrato utilizando *extrairChamadas*. Adicionalmente, rastreia o uso de variáveis de ambiente nativas da EVM, como *msg.sender* e *msg.value*, registrando-as formalmente no dicionário de variáveis globais para que atuem como Oráculos implícitos na rede.

O método *extrairOperacoes* é responsável por identificar mutações de estado de variáveis globais. Ele utiliza uma Expressão Regular (RegEx) estruturada em três grupos de captura: a variável de destino (*L-value*), o operador de mutação ($=$, $+=$, $-=$) e o operando à direita da equação (*R-value*). O método também intercepta operações físicas de vetor, convertendo chamadas nativas de *.push()* e *.pop()* em operações matemáticas de adição e subtração. Para cada operação válida identificada, instancia-se um novo objeto da classe *OperacaoSolidity*.

Em complemento, o método *extrairChamadas*, auxiliado pelo filtro *deveIgnorarChamada*, rastreia o grafo de execução do contrato. Ele elimina falsos positivos ignorando palavras reservadas de controle de fluxo, erros customizados e primitivas nativas de vetor (*push*, *pop*, *type*), focando no mapeamento de chamadas reais entre subfunções para preservar o encadeamento de estado.

Por fim, as funções *visitEnumDefinition* e *visitStructDefinition* são encarregadas de capturar Tipos Definidos pelo Usuário (UDT). O *Visitor* mapeia os domínios finitos de valores das enumerações e extrai os campos das estruturas (utilizando *LinkedHashMap* para preservar a ordem exata de declaração).

3 Transformação do Solidity para Rede de Petri

3.1 Classe GerenciadorVariaveis

Durante a etapa de transformação do contrato para RPC, cada transição exige um escopo local e isolado. As variáveis utilizadas nas expressões dos arcos não podem colidir entre diferentes transições, mas devem manter coerência entre os arcos de leitura (entrada) e de mutação (saída). A classe responsável por orquestrar essa memória temporária e converter identificadores em símbolos é a *GerenciadorVariaveis.java*.

A estrutura base de armazenamento da classe é o atributo *mapaVariaveis*, implementado através de um `LinkedHashMap<String, String>`. A escolha desta estrutura se dá pois ela preserva a ordem cronológica de inserção dos elementos e permite ordená-los pelo comprimento da chave no momento da compilação das equações. Isso impede falhas de substituição parcial em expressões complexas. Evitando, por exemplo, que a substituição de uma variável curta corrompa o nome de um identificador mais longo que a contenha como substring.

A alocação principal é feita pela função *getOuCriarVariavel*. Este método consome sequencialmente a Tabela ASCII (*A, B, C...Z, A1, B1...*) para designar símbolos formais às variáveis do contrato. Quando uma variável global, parâmetro ou estrutura de índice do Solidity é requisitada pela primeira vez

durante a montagem dos arcos de entrada de uma transição, o método aloca o próximo caractere disponível no alfabeto e o associa ao respectivo *ColorSet*.

Por fim, a classe atua de maneira integrada na normalização de matrizes e mapeamentos multidimensionais. Quando variáveis complexas compartilham a mesma estrutura de indexação (por exemplo, chaves idênticas de acesso em `mapping(uint8 => mapping(uint8 => uint256))`), o *GerenciadorVariaveis* identifica a paridade das chaves e garante que os arcos gerados compartilhem as mesmas variáveis, viabilizando a correlação de *tokens* simultâneos.

3.2 Construção da Rede: Classe *RPCBuilder*

A classe *RPCBuilder.java* serve para transformar de fato o contrato em uma rede de Petri. Sua responsabilidade consiste em consumir o agregador de metadados *ListasInfo* extraído pelo *SolidityVisitor*, e converter a semântica do contrato inteligente para as normas formais de especificação em SML requeridas pelo CPN Tools. Para orquestrar a rede, a classe administra internamente listas de entidades (*Lugar*, *Transicao* e *Arco*), dicionários de acesso rápido (*lugaresVariaveis* e *transicoesFunc*) e uma estrutura de controle de escopo léxico (*gerenciadoresLocais*), que mapeia e fixa uma instância exclusiva de *GerenciadorVariaveis* para cada transição do modelo.

O ponto de entrada é o método *construirRPC*, que define a ordem para a montagem da rede. O algoritmo inicia pela criação dos lugares, pelas funções *criarLugaresVariaveis* e *criarLugaresOraculos*. Cada variável global de estado é instanciada como um Lugar, cuja tipagem é submetida ao conversor *mapearTipoParaColorSet*. Este conversor identifica mapeamentos (*mappings*) e vetores multidimensionais para sintetizar *ColorSets* (por exemplo, convertendo `mapping(Cliente => uint256)` na notação `CLIENTExUINT`). Funções com visibilidade *public* ou *external* geram Lugares Oráculos para parametrizar entradas da *blockchain*, enquanto o construtor do contrato recebe tratamento especial: é isolado em um Lugar com conjunto de cores *BOOL* e marcação inicial restrita a *1>true*, garantindo seu disparo único na inicialização do sistema.

Em seguida, as funções são transformada em transições pelo método *criarTransicoesFunc*, que instancia a classe *Transicao* para cada função do contrato. A assinatura de cada transição é formulada pelo método *construirGuardExpressao*, o qual agrega restrições de acesso (como o modificador *onlyOwner*) e validações de barreira (*require* e *assert*) em uma única expressão booleana combinada por portas lógicas *AND*. Para o construtor do contrato, a guarda é fixada como *true*.

A função *mapearVariaveisAFuncoes* serve como conexão entre o estado e os eventos é feita pelo método *.* Esta rotina varre as operações, condicionais e chamadas de cada função para catalogar todas as variáveis globais acessadas ou mudadas. Para evitar anomalias de indexação na ordem de geração, a matriz de dependência emprega estruturas *LinkedHashSet*, para manter a ordem na leitura de variáveis. A partir desse mapeamento, o método *criarArcosFluxoDados* faz a criação de arcos de entrada, que transferem o estado de um Lugar para a Transição por meio de tuplas geradas pelo *GerenciadorVariaveis* (como (C, E)),

enquanto o método providencia o pré-carregamento de sufixos de valor (`._val`) para que arranjos bidimensionais compartilhem ponteiros.

O método *traduzirOperacaoParaRPC* transforma as mutações feitas no contrato Solidity (`=`, `+=`, `-=`) em equações de transição de estado em SML. Uma operação de adição como `balances[receiver] += amount` é convertida para a tupla de gravação (`A, E + B`) no arco de saída. O método *aplicarCondicionalisRPC* ao identificar comandos estruturados como `if_revert`, inverte a condição (aplicando o utilitário *inverterLogica*) para gerar bifurcações na notação SML: quando a condição de reversão é acionada, o fluxo descarta os *tokens* usando `-> NULL`, preservando a integridade das marcações da rede em cenários de falha na transação.

A função *criarArcosDeChamadas* identifica quando uma transição principal aciona uma subfunção interna, o compilador rastreia os identificadores paramétricos na tabela ASCII para traçar arcos de chamada (como (A, B)), conectando o evento chamador ao Lugar Oráculo da função destino e mantendo a rastreabilidade integral do grafo de execução na Rede de Petri Colorida.